

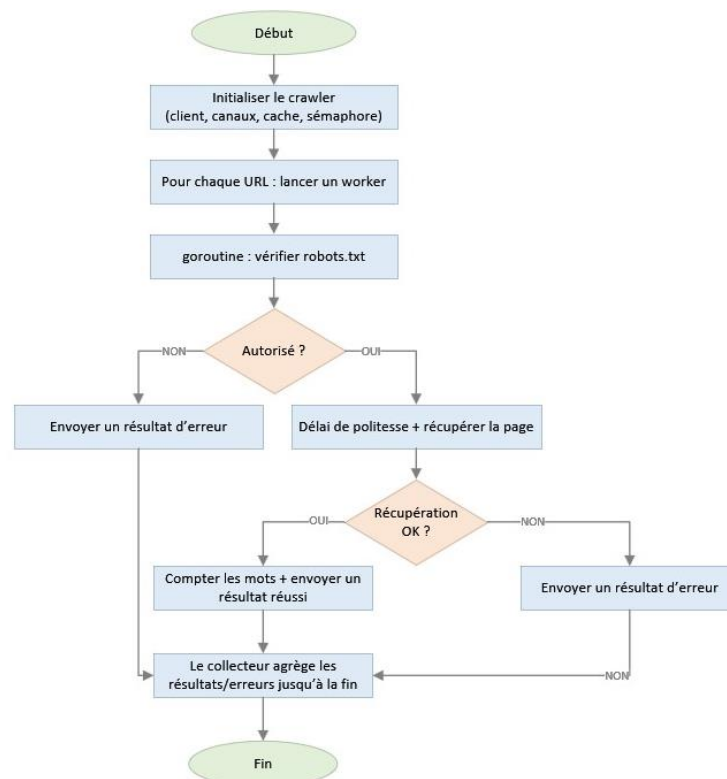
TN6 – Robot d'exploration Web concurrent

1. Implémentation

1.1 Architecture générale

Le programme repose sur neuf fonctions à responsabilité unique, qui propagent les erreurs explicitement plutôt que de paniquer. Cette conception garantit qu'une URL défaillante n'interrompt pas l'exploration des autres, qu'il s'agisse d'un timeout HTTP, d'un code de statut non-200, d'une lecture interrompue ou d'une URL malformée. CrawlURLs orchestre l'exploration concurrente en lançant une goroutine par URL, bornée par un sémaphore. La logique principale est extraite dans `run()` pour permettre les tests unitaires sans modifier le comportement en production. La figure 1 illustre le flux d'exécution complet pour une URL, des deux points de décision jusqu'à l'agrégation des résultats par le consommateur unique.

Figure 1 –Flux d'exécution de `crawlURL()`



1.2 Gestion de la concurrence

Plutôt qu'un mutex sur la map des résultats, l'implémentation retenue utilise le patron producteur-consommateur unique, où les goroutines déposent leurs résultats dans un canal bufferisé (*chan CrawlResult*) et une goroutine consommatrice unique les lit avec `for result := range ch`. Puisqu'une seule goroutine écrit dans la map et le compteur total, toute course aux données est structurellement impossible.

Un *sync.RWMutex* protège le cache *robots.txt* par hôte, où plusieurs goroutines lisent et écrivent simultanément. Le patron de double vérification, consistant à effectuer une lecture partagée, détecter un défaut de cache puis acquérir un verrou exclusif avant d'écrire, garantit l'exactitude sans bloquer les lectures concurrentes.

Le parallélisme est borné par un sémaphore implémenté comme un canal bufferisé de capacité `maxGoroutines`. Chaque goroutine acquiert un jeton au démarrage et le libère via `defer`, évitant de saturer les ressources réseau ou le planificateur Go.

1.3 Respect de robots.txt

checkRobotsAllowed() délègue la récupération à *fetchRobots()*, qui retourne *nil* si le fichier est absent ou inaccessible, autorisant l'exploration par défaut conformément à la convention standard. Les résultats sont mis en cache par hôte pour éviter des requêtes répétées vers */robots.txt*.

Un délai de politesse de 100 ms (*politenessDelay*) est appliqué avant chaque requête principale pour limiter la fréquence des accès aux serveurs explorés. Il est désactivé dans les bancs d'essai pour isoler l'effet réel du parallélisme.

1.4 Comptage des mots

countWordsHTML() lit le flux de jetons HTML séquentiellement avec *golang.org/x/net/html*, ce qui garantit un comportement correct face à du HTML malformé, contrairement à une expression régulière. Les balises *<script>*, *<style>* et *<noscript>* sont ignorées via un drapeau booléen activé à l'ouverture et réinitialisé à la fermeture. Les entités HTML sont décodées automatiquement par le tokeniseur.

2. Tests unitaires

Le fichier *crawler_test.go* contient 28 tests unitaires et 3 bancs d'essai, atteignant une couverture de code de 100 %. Tous les tests s'appuient sur *httptest.NewServer* pour éliminer toute dépendance au réseau réel et garantir des résultats reproductibles. Une exception a nécessité l'utilisation directe de *net.Listen* pour simuler des comportements impossibles à reproduire avec un serveur HTTP standard, notamment un serveur qui déclare un corps de 1 000 octets, mais ferme la connexion après 7 octets, seul moyen de couvrir le chemin d'erreur de *io.ReadAll* dans *fetchRobots*.

Les tests couvrent non seulement les chemins nominaux, mais aussi les cas limites les plus difficiles à provoquer. Le timeout de 10 secondes est vérifié par un serveur qui ne répond jamais, prouvant que le client HTTP le respecte effectivement. Le test sur le code 404 existe parce que *http.Client* ne retourne pas d'erreur sur les codes 4xx par défaut, la fonction devant vérifier explicitement le statut. Pour *checkRobotsAllowed*, un octet nul dans l'URL est le seul moyen fiable de forcer *url.Parse* à retourner une erreur et d'atteindre la branche *return false*.

La testabilité de *main()* a été assurée en exposant *mainURLs* comme variable de paquet, que les tests substituent temporairement par une URL locale avant de restaurer la valeur originale via *defer*. Ce patron d'injection légère permet de couvrir le point d'entrée sans appel réseau réel. De même, *run()* est testée avec trois scénarios distincts, soit le succès, les erreurs uniquement et les résultats mixtes, pour couvrir ses deux branches d'affichage indépendantes.

3. Analyse des performances

3.1 Protocole et résultats

Les données de test sont générées hors de la boucle *b.N* et *b.ReportAllocs()* confirme l'absence d'allocation parasite dans la boucle de mesure. *b.ResetTimer()* est appelé dans *BenchmarkCrawlGoroutinesMultiServer* et *BenchmarkCountWordsHTML* pour exclure le temps d'initialisation des serveurs de test. Le délai de politesse est désactivé (*politenessDelay* = 0) pour isoler l'effet du parallélisme. Les benchmarks ont été exécutés avec *go test -bench=Benchmark -benchmem -run=^\$ -count=6* sur un Intel i5-10300H à 2,50 GHz (Windows/amd64).

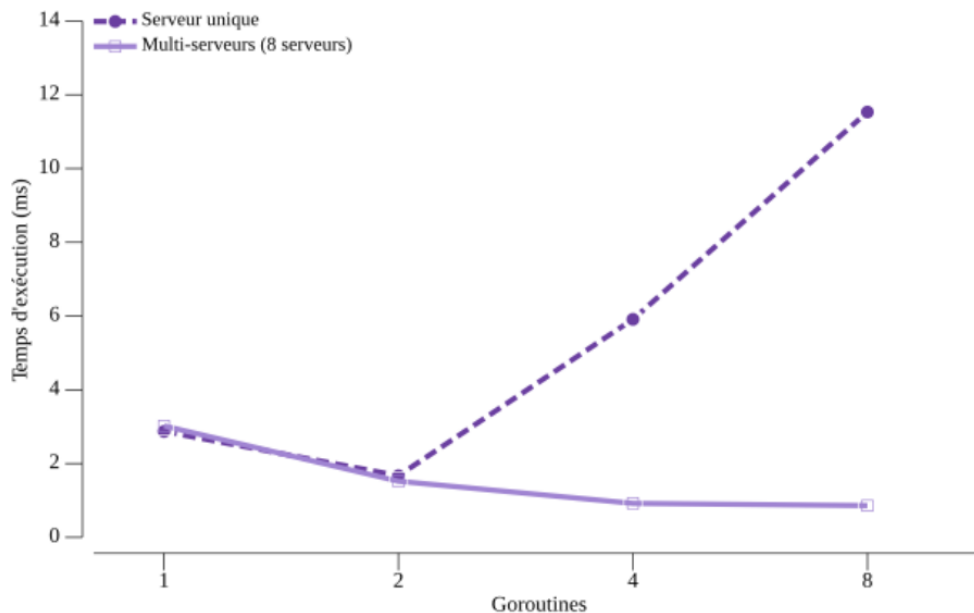
Deux scénarios distincts séparent deux sources de contention indépendantes. *BenchmarkCrawlGoroutines* compare 1, 2, 4 et 8 goroutines sur 8 URLs vers un serveur unique, introduisant une contention côté serveur. *BenchmarkCrawlGoroutinesMultiServer* reproduit la même comparaison avec 8 serveurs distincts pour mesurer le gain de parallélisme réel. *BenchmarkCountWordsHTML* mesure isolément le tokeniseur HTML sur une page de ~1 900 mots.

Tableau 2 – Résultats des benchmarks de crawl (benchstat, count=6)

Configuration	Temps (ms)	Variance	Accélération vs 1G
Serveur unique, 1 goroutine	2,87	± 9 %	1,00×
Serveur unique, 2 goroutines	1,67	± 9 %	1,72×
Serveur unique, 4 goroutines	5,91	± 87 %	0,49×
Serveur unique, 8 goroutines	11,53	± 13 %	0,25×
Multi-serveurs, 1 goroutine	3,02	± 8 %	1,00×
Multi-serveurs, 2 goroutines	1,52	± 5 %	1,99×
Multi-serveurs, 4 goroutines	0,92	± 1 %	3,28×
Multi-serveurs, 8 goroutines	0,86	± 22 %	3,52×

La figure 2 illustre visuellement le croisement des deux courbes à 2 goroutines, point à partir duquel les comportements divergent selon la source de contention.

Figure 2 – Temps d'exécution selon le nombre de goroutines



3.2 Analyse

Avec un serveur unique, les performances se dégradent au-delà de 2 goroutines. La variance de $\pm 87\%$ à 4 goroutines et la régression à 8 goroutines s'expliquent par le traitement séquentiel des connexions dans `httptest.NewServer`, qui transforme le surcroît de goroutines en surcoût de coordination. Ce comportement est un artefact du banc d'essai, non un défaut du crawler, mais il illustre un phénomène réel lorsque plusieurs URLs partagent le même hôte.

Avec 8 serveurs distincts, le gain atteint $3,52\times$. La progression de 3,02 ms à 0,86 ms illustre le rendement décroissant attendu, où la portion séquentielle incompressible du programme limite les gains à mesure que le nombre de goroutines augmente. La variance de $\pm 22\%$ à 8 goroutines contre $\pm 1\%$ à 4 goroutines confirme que le planificateur Go et le coût de synchronisation des canaux deviennent les facteurs limitants.

BenchmarkCountWordsHTML mesure $173\ \mu\text{s} \pm 8\%$ et 48 Ko pour 204 allocations, confirmant que le tokeniseur ne constitue pas un goulot d'étranglement face à une latence réseau qui le dépasse d'un facteur supérieur à 10.

La limite de performance du crawler est la latence réseau, non le parsing HTML ni la synchronisation des goroutines. L'accélération de $3,52\times$ en scénario multi-hôtes confirme que l'architecture concurrente exploite efficacement le parallélisme disponible.

4. Défis et optimisations

Sans mise en cache de robots.txt, chaque URL aurait déclenché une requête supplémentaire vers /robots.txt, doublant le trafic réseau. Un cache par hôte protégé par un `sync.RWMutex` élimine ces requêtes redondantes. Le patron de double vérification, lecture partagée puis verrou exclusif en cas de défaut de cache, garantit l'exactitude sans bloquer les lectures concurrentes.

Le délai de politesse de 100 ms, indispensable en production, aurait artificiellement masqué l'effet du parallélisme dans les bancs d'essai. L'exposition de `politenessDelay` comme variable de paquet permet de le désactiver en test sans modifier le comportement en production.

Atteindre 100 % de couverture a exigé de simuler des comportements impossibles avec `httptest.NewServer`, notamment une connexion TCP fermée avant la fin du transfert, qui a nécessité l'utilisation directe de `net.Listen` pour contrôler précisément le comportement réseau simulé, et un octet nul dans l'URL, qui force `url.Parse` à retourner une erreur sans aucune infrastructure réseau. La testabilité de `main()` a été assurée via la variable de paquet `mainURLs`, substituable dans les tests sans appel réseau réel.

5. Conclusion

Ce projet démontre comment la robustesse, la concurrence structurée et la mesure rigoureuse se combinent pour produire un crawler fiable et performant. Le patron producteur-consommateur unique élimine toute course aux données, le sémaphore bufferisé contrôle le degré de parallélisme et les 28 tests unitaires atteignent une couverture de 100 %. Les benchmarks confirment une accélération de $3,52\times$ en scénario multi-hôtes et un parsing HTML de $173\ \mu\text{s}$ pour 48 Ko alloués. La conformité à robots.txt avec cache par hôte et délai de politesse configurable complète une implémentation correcte, performante et éthiquement responsable.